Part01

1-

```
1  // int x = 10;
2  /*int y = 20;
3  /int sum = x + y;
4  Console.WriteLine(sum);*/
```

2- CTRL + /

3- Assigning an intger variable a string and using undeclared variable

4- Runtime error is happening while the code is running something like “num/0”

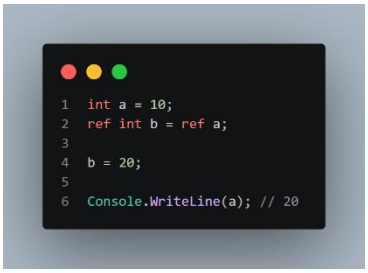
And logical error is something wrong in the logic of the code it is something like “1+1=3”

5-

```
1  string name;
2  int age;
3  int salary;
4  bool isStudent;
```

6- Because it makes the code easy to read

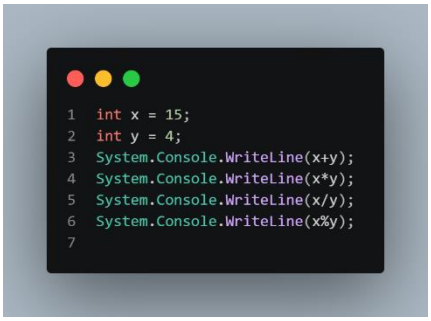
7-



```
1  int a = 10;
2  ref int b = ref a;
3
4  b = 20;
5
6  Console.WriteLine(a); // 20
```

8- **Value type take a copy of the value to use in another different address in the memory Reference type take the value itself with the same address and use it with another variable**

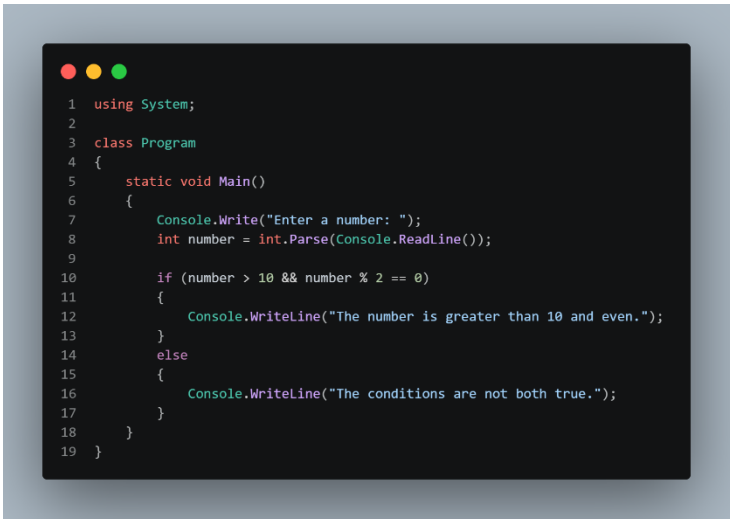
9-



```
int x = 15;
int y = 4;
System.Console.WriteLine(x+y);
System.Console.WriteLine(x*y);
System.Console.WriteLine(x/y);
System.Console.WriteLine(x%y);
```

10- **It will be 2 because 2/7=0 so the reminder is 2**

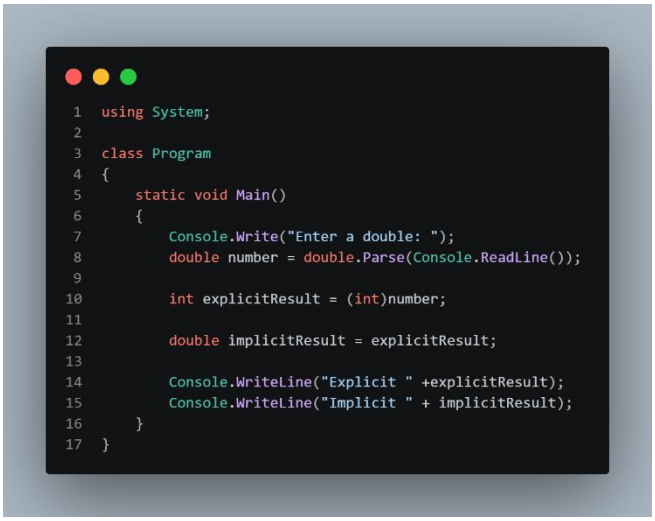
11-



```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int number = int.Parse(Console.ReadLine());

        if (number > 10 && number % 2 == 0)
        {
            Console.WriteLine("The number is greater than 10 and even.");
        }
        else
        {
            Console.WriteLine("The conditions are not both true.");
        }
    }
}
```

12- **&& is used to add more that 1 condition in “if” that all must achieve while & is a bitwise operator that Anding each bit in both sides**

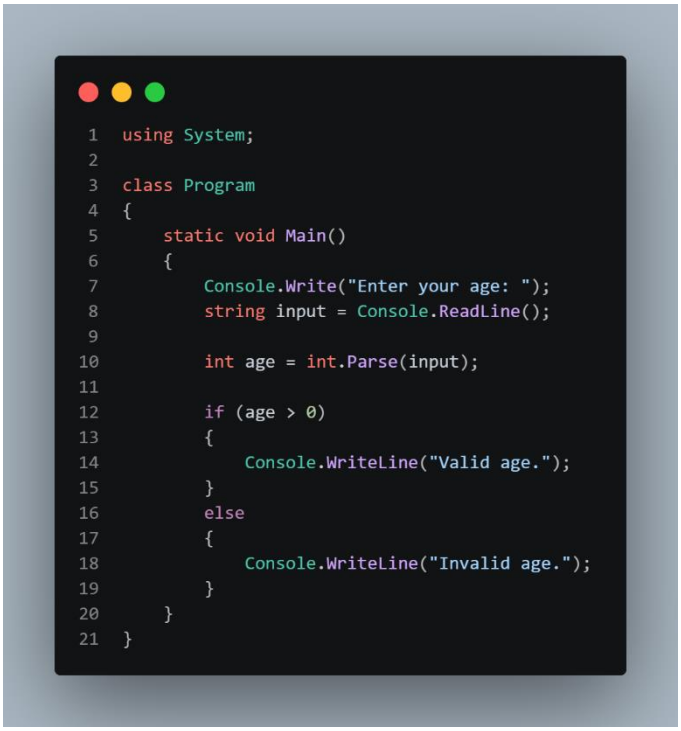
13-



```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter a double: ");
        double number = double.Parse(Console.ReadLine());

        int explicitResult = (int)number;

        double implicitResult = explicitResult;

        Console.WriteLine("Explicit " +explicitResult);
        Console.WriteLine("Implicit " + implicitResult);
    }
}
```

14- Because double might has bigger value than int so we need to use explicit casting so no data can be missed

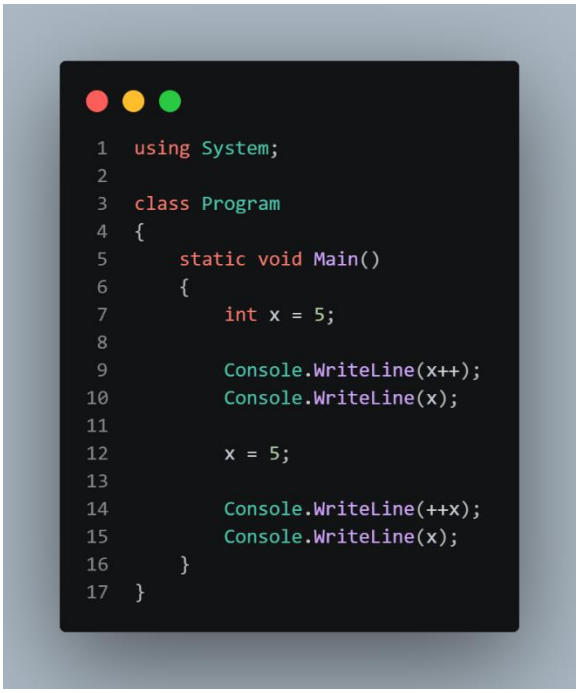
15-



```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter your age: ");
        string input = Console.ReadLine();

        int age = int.Parse(input);

        if (age > 0)
        {
            Console.WriteLine("Valid age.");
        }
        else
        {
            Console.WriteLine("Invalid age.");
        }
    }
}
```

16- If the use enter characters instead of number so we can convert it and I can handle it using int.tryparse() that can check first if it can be converted or not

17-



```
using System;

class Program
{
    static void Main()
    {
        int x = 5;

        Console.WriteLine(x++);
        Console.WriteLine(x);

        x = 5;

        Console.WriteLine(++x);
        Console.WriteLine(x);
    }
}
```

18 – it will be 12 because the ++x increment first then use the value and x++ use the value and then increment

Part02

2-

Compiled languages: The code is converted into machine code before the program runs.
Example: C++

Interpreted languages: The code is executed by an interpreter while the program is running.
Example: Python

What about C#?
C# is compiled, but not directly into machine code. The C# compiler converts the code into IL (Intermediate Language), then the .NET runtime uses JIT to convert it into machine code when the program runs.

In short:

C++ → Source Code → Machine Code
C# → Source Code → IL → JIT → Machine Code
Python → Source Code → Interpreter → Execution

3- Implicit: Conversion happens automatically when it is safe.

Explicit: You manually convert the value using a specific type.

Convert: Used to convert between different data types.

Parse: Used mainly to convert a string into a specific data type